

This problem set is due by 11:59pm on **Wednesday, March 12**. It should be submitted through Canvas. All submissions should be typeset using Latex.

Problem 1 (Gradient-Free Policy Search) *In this problem, we will use approaches for gradient-free or model-free policy search to solve the perching problem.*

- a) (3 points) *Explore the code in `glider_gradient_free.m`. First implement the function “`sampleTrajectoryCosts`”. In this function we “roll-out” trajectories and evaluate the instantaneous and total cost.*
- b) (2 points) *We are going to start by implementing model-predictive path-integral (MPPI) control. Does the glider dynamics model, as given, allow MPPI to reason about physical uncertainty? If not, how would the dynamics model need be modified to allow this? What restrictions are there on this uncertainty model?*
- c) (5 points) *Implement model-predictive path-integral (MPPI) control by filling in the “MPPI” function. Assume a special case of the dynamics where*

$$dX_t = f(X_t)dt + g(X_t, t)(u(X_t, t) + \frac{1}{\sqrt{\rho}} \frac{\epsilon}{\sqrt{dt}})dt \quad (1)$$

and ϵ is Gaussian noise with a variance of 10. Assume the noise is scaled by $\sqrt{\rho}$, where $\rho = 10000$. Plot the resulting position trajectory and cost per iteration. Submit your code and plots.

- d) (5 points) *Implement cross-entropy trajectory optimization by filling in the “`cross_entropy`” function. Assume your policy parameters are sampled from a Gaussian distribution with a diagonal covariance. Use trajectory samples to select an elite set of costs. Then use maximum-likelihood estimation to iteratively refine the parameters of the policy distribution using this elite set. Plot the resulting position trajectory and cost per iteration. Submit your code and plots.*
- e) (2 points) *Compare the quality and computational requirements of the different gradient-free approaches. How does the number of total roll-outs compare with the cost achieved?*

Problem 2 (Fitted Value Functions) *In this problem, we will explore fitted value iteration.*

- a) (5 point) *Examine and run the file `regularized_least_squares_test.m`. This file fits a cubic polynomial to data using least squares. Now consider the file `continuous_grid_world_no_obstacles.m`. Choose sample points for the value iteration algorithm (fill in the code), and use your understanding of `regularized_least_squares_test.m` to approximate the cost-to-go using both a two-dimensional linear and a two-dimensional quadratic cost-to-go. To approximate the cost-to-go, use the function “`fit_cost_to_go`”. Then fill in the functions “`fit_linear`”, “`fit_quadratic`” and the functions for the associated nonlinear basis functions.*

- b) (5 points) Plot the resulting linear and quadratic cost-to-gos in three-dimensions. Does the linear cost-to-go converge to the optimal cost-to-go? What about the quadratic cost-to-go? Explain why the cost-to-gos do or do not converge to the optimal cost-to-go. If either of the cost-to-gos do not converge, find a cost-function that enables the system to reach the goal but also allows convergence to the optimal cost-to-go. Submit your plots and code.

Problem 3 (Online Approximate Dynamic Programming) In this problem, we explore methods for online dynamic programming that leverage the temporal-difference (TD) update.

- a) (2 points) Complete the code `grid_world_td_learning.m`. Fill in the number of iterations, learning rate, and provide an approach for selecting the initial state. HINT: Functions exist in the code to support selecting a random state in grid world.
- b) (3 points) Fill in the code for the temporal difference update in function "td_update" as well as the code for selecting the greedy action in "get_action". Run the algorithm until it converges and plot your results, including the cost-to-go, the 2-norm of the cost-to-go with time, and the resulting trajectory from executing the policy starting from $x = [1, 1]$.
- c) (2 points) Now complete the code in the file `grid_world_sarsa_qlearning.m`. Fill in the code for the "td_update" for both the SARSA algorithm and the Q-learning algorithm.
- d) (3 points) For the "get_action" function, now implement an ϵ -greedy approach. Choose your ϵ , run the algorithms to convergence and plot the resulting the cost-to-go, the 2-norm of the cost-to-go with time, and the resulting trajectory from executing the policy starting from $x = [1, 1]$. Describe and explain any difference you see in the results.
- e) (5 points) Now change the grid-world environment to "use_pits". Run the SARSA and Q-learning algorithms again to convergence, and plot the resulting the cost-to-go, the 2-norm of the cost-to-go with time, and the resulting trajectory from executing the policy starting from $x = [1, 1]$. Describe/explain any differences. Submit code and plots.

Problem 4 (Neural Fitted Value Iteration) In class, we discussed fitted value iteration. In this problem, we will explore fitted value iteration using neural networks.

- a) (3 points) In this problem, we will be leveraging the MATLAB Deep Learning Toolbox. Explore the code `brick_value_function_fit_test.m` and familiarize yourself with the neural network function approximation tools. Choose layer output sizes, the maximum number of epochs, and the mini batch size so that you can adequately approximate the cost-to-go computed in problem set 1 for the double integrator with a quadratic cost. Submit your code and plots.

- b) **(2 points)** Similarly, complete the code `pend_value_function_fit_test.m` so that the neural network cost-to-go adequately represents the provided cost-to-go from problem set 1. Submit your code and plots. How is the number of weights required to approximate the pendulum cost-to-go different from the number required for the brick cost-to-go?
- c) **(10 points)** Based on your understanding of the MATLAB deep learning tools, and the required number of neural network parameters for good cost-function approximation, fill in the code in `brick_vi_fitted.m`. In particular, define, configure, and initialize your neural network. Then, use your neural network to do fitted value iteration to approximate the cost-to-go. Using the state-space samples in the code, apply the Bellman update to evaluate the cost-to-go at the sample points. Then train the neural network to update your approximation of the cost-to-go. Run the algorithm to convergence. Submit your code, and plots of your cost-to-go and resulting state-space trajectory.
- d) **(5 points)** Carry out fitted value iteration for the simple pendulum. To do this complete the code `pend_vi_fitted.m`. Submit your code, and plots of your cost-to-go and resulting state-space trajectory. Remember to account for angle wrapping.